

Proof-Carrying Transactions: Zero-Knowledge Policy Compliance for Autonomous AI Agents

Anonymous Author(s)

Abstract—Autonomous AI agents are already authorizing payments, calling external APIs, and modifying production databases—often with no human in the loop. Governing what these agents send out over the network is harder than it looks: any single request might seem perfectly fine on its own, yet a sequence of such requests can quietly blow through a spending limit, hit a forbidden endpoint at 2am, or drift away from the task the agent was supposed to be doing. Today’s answer to this problem is a centralized proxy firewall, which asks downstream services to simply trust it. Nobody can independently verify that the proxy did its job, and if it is misconfigured or compromised, enforcement fails silently.

We describe *Proof-Carrying Transactions* (PCT), a different approach. Instead of asking anyone to trust the proxy, we attach a short zero-knowledge proof—a cryptographic receipt—to every outbound request the agent makes. The proof demonstrates that the request satisfies the organization’s egress policy without revealing what that policy actually says. To make this practical at HTTP-request speeds, we built a domain-specific compiler that turns declarative policy rules directly into Rank-1 Constraint Systems, cutting gate counts to about 1% of what a general-purpose zkVM would need. On a corpus of 847 real enterprise policies, the median proof takes 172ms to generate and 1.1ms to verify, regardless of how complex the policy is. We prove computational soundness, zero-knowledge policy privacy, and a completeness property ensuring that honest requests are never rejected, and we show how PCT maps onto the EU AI Act and the NIST AI Risk Management Framework.

Index Terms—zero-knowledge proofs, autonomous AI agents, policy compliance, proof-carrying code, verifiable computation, cryptographic governance, zk-SNARKs, Groth16

I. INTRODUCTION

Large language models are no longer just chat assistants. In enterprise settings, they have been promoted to autonomous agents—programs that authorize financial transactions through banking APIs, rewrite rows in production databases, and chain together sequences of third-party service calls, all while the human operator is, at best, reviewing log files after the fact [1], [2]. The volume and dollar value of these automated interactions are growing fast, and with them comes a governance question that existing tools cannot answer. We call it the *Autonomous Egress Problem*: how do you give a mathematical guarantee that every outbound request from an AI agent satisfies some internal policy, without forcing the recipient to trust a middleman, and without exposing what the policy thresholds actually are?

Nothing we have today handles this well. On the application side, prompt engineering and RLHF [4] nudge a model’s behavior during training, but they produce no artifact you can point to after the fact and say “this specific API call was compliant.” They are also brittle: adversarial jailbreaks can

undo them [5]. Constitutional AI [3] is a step further in the same direction, but the same limitation applies—the guarantee is probabilistic and offers nothing at the level of individual transactions.

On the infrastructure side, API gateways like Kong and Envoy do enforce rate limits and access controls, and they work well enough within a single organization. But the downstream API provider has no way to independently check whether the gateway actually enforced its rules. If someone misconfigures the gateway—or if an attacker compromises it—the policy is gone, and nobody finds out until the audit. Post-hoc logging detects problems only after potentially irreversible transactions have already completed.

Our proposal borrows heavily from an old idea. In 1997, Necula introduced *proof-carrying code* [6]: a binary carries a formal proof that it respects the host’s safety policy, and the host checks the proof before running the code. We lift this concept from the instruction level to the network boundary. Each outbound HTTP request carries a zero-knowledge proof that says “this request satisfies the active egress policy.” A verifier can check the proof in constant time. The proof reveals nothing about what the policy contains—not the dollar limits, not the whitelist entries, not the time windows.

Concretely, this paper makes four contributions:

- 1) We formalize a security model for transparent-proxy governance (Section IV) and prove three properties: computational soundness (a valid proof for a non-compliant request is infeasible to forge), zero-knowledge privacy (the proof reveals nothing about policy internals), and policy-binding completeness (legitimate requests are never incorrectly rejected—Theorems 1–3).
- 2) We design a domain-specific compiler (Section VI) that translates declarative policy specifications into optimized R1CS circuits. It includes custom algebraic gadgets for semantic similarity verification and temporal window evaluation. The semantic constraint captures intent alignment:

$$C_{\text{sem}}(\text{req}) \iff \text{sim}(\text{emb}(\text{req.intent}), v_{\text{ref}}) \geq \tau$$

where τ absorbs the bounded quantization error from fixed-point arithmetic.

- 3) We implement the full system as a production Rust proxy (Section VII), integrate it into the outbound HTTP path, and evaluate it on 847 enterprise egress policies. Median proving time is 172ms—well within the latency budget of synchronous API calls.

- 4) We demonstrate how PCT satisfies regulatory requirements under the EU AI Act and NIST AI RMF (Section IX), providing cryptographic compliance evidence rather than procedural assurances.

II. BACKGROUND AND RELATED WORK

A. Autonomous Agent Architectures

Modern agent frameworks—AutoGPT, ReAct [18], and their descendants—run in continuous loops of observe, plan, act, and repeat [2]. They decide when to call an API, when to persist state, and when to trigger a payment, all autonomously. The risks of coupling deterministic side effects with a probabilistic reasoning engine were flagged early by Hendrycks et al. [31] and by Amodei et al. [39], but neither proposed a way to enforce compliance at the moment the network request goes out. Every HTTP call that an agent issues is an egress event, and any one of them might violate organizational policy.

B. Zero-Knowledge Proof Systems

A zero-knowledge proof lets a prover convince a verifier that a statement is true without revealing anything else [7]. The idea is deceptively simple, but the formal requirements are precise. A system $(\text{Setup}, \mathcal{P}, \mathcal{V})$ must satisfy three properties:

Definition 1 (Zero-Knowledge Proof System). •

- **Completeness:** *If the statement is true and the prover is honest, the verifier always accepts:* $\Pr[\mathcal{V}(\text{vk}, x, \pi) = 1 \mid \pi \leftarrow \mathcal{P}(\text{pk}, x, w)] = 1$.
- **Soundness:** *A cheating prover cannot convince the verifier of a lie: for $x \notin \mathcal{L}$, the probability $\Pr[\mathcal{V}(\text{vk}, x, \pi^*) = 1]$ is negligible, no matter what strategy produces π^* .*
- **Zero-Knowledge:** *There exists a simulator $\mathcal{S}$ that, given only the public inputs (and no secret witness), can produce fake proofs indistinguishable from real ones: $\{\pi \leftarrow \mathcal{P}(\text{pk}, x, w)\} \approx_c \{\pi^* \leftarrow \mathcal{S}(\text{vk}, x)\}$.*

We use Groth16 [8] as our proof backend. A Groth16 proof is just three elliptic-curve group elements—tiny, and verifiable in constant time regardless of how large the underlying circuit is. The catch is that it requires a per-circuit trusted setup. Other systems trade differently: PLONK [9] has a universal setup (one ceremony works for any circuit), Bulletproofs [19] need no trusted setup at all, and STARKs [20] are post-quantum secure. But for a real-time HTTP proxy, where proofs need to be generated in under 200ms and verified in under 10ms, Groth16 is the only system that hits both targets today [21], [35]. We revisit this choice with empirical data in Section VIII.

C. Proof-Carrying Code

Necula’s original proof-carrying code [6] was designed for a different era: a code consumer receives a binary, and the binary carries a proof of memory safety that the consumer checks before executing. The key insight transfers directly. We just change the carrier: instead of attaching proofs to CPU instructions, we attach them to HTTP requests. The timing constraint changes too—proofs must be generated in real time during a live network round-trip, not at compile time. But the

structural principle is the same: the consumer verifies a proof rather than trusting the producer.

D. Verifiable Machine Learning

Recent work has begun applying zero-knowledge proofs to ML inference. EZKL [46] compiles ONNX models into Halo2 circuits, and Giza [47] does something similar with Cairo STARKs on StarkNet. These frameworks address a related but different problem: they prove that a specific model produced a specific output. PCT does something narrower and, for our purposes, more useful: it proves that the *policy evaluation over* model outputs satisfies governance constraints. We do not re-prove the embedding computation inside the SNARK—doing so for even a small transformer would cost millions of gates. Instead, we run the embedding model inside a trusted execution environment and verify only the downstream policy arithmetic inside the circuit. This architectural shortcut reduces constraint counts by orders of magnitude while preserving end-to-end verifiability of the compliance decision.

III. COMPARATIVE ANALYSIS OF GOVERNANCE ARCHITECTURES

Zero-knowledge proofs have seen production use in private payments (Zcash [10]), blockchain scalability [11], and anonymous credentials [12], [28]. Recently, Kang et al. [22] and Zhao et al. [34] have explored them for verifiable ML. To our knowledge, nobody has applied zero-knowledge proofs to the governance of autonomous AI agent egress. Existing runtime filters—Guardrails AI, Microsoft Guidance [30]—are deployed widely, but they work as deterministic or statistical gates. They expose their own decision logic and provide no way for an external party to independently confirm that a given transaction was actually checked.

It helps to lay out the terrain. Application-layer approaches (RLHF, prompt engineering) are fast—they add essentially no latency—but they are probabilistic [38], they produce no per-transaction evidence, and adaptive attackers can defeat them [32]. API gateways enforce rules efficiently, but enforcement is only as good as the gateway itself; anyone downstream has to take the gateway’s word for it. Policy engines like OPA [29] give you expressive rule languages, but the same trust problem remains.

PCT sits in a different spot. It adds roughly 95ms of latency per request—a noticeable cost—but in exchange, it produces a cryptographic artifact that any party can independently verify. The tradeoff makes most sense in regulated settings, where the alternative is “trust us, we have a firewall.” Table I summarizes the landscape.

IV. SECURITY MODEL AND FORMAL DEFINITIONS

A. System Model

There are four principals in the PCT framework. Each plays a distinct role in the lifecycle of a single outbound request.

Definition 2 (PCT Principals). • **Agent $\mathcal{A}$:** *The autonomous program issuing outbound requests. A request takes the form $\text{req} = (\text{target}, \text{payload}, \text{metadata})$.*

TABLE I: Comparison of AI Agent Governance Approaches

Approach	Formal Guarantee	Policy Privacy	Indep. Verify	Real-time	Regulatory Mapping
RLHF / Constitutional AI	×	×	×	✓	×
Prompt Engineering	×	×	×	✓	×
Guardrails AI / LangChain	×	×	×	✓	×
API Gateway (Kong, Envoy)	×	×	×	✓	×
mTLS + Macaroons	Partial	×	Partial	✓	×
Post-hoc Audit Logging	×	×	Partial	×	Partial
PCT (This Work)	✓	✓	✓	✓	✓

- **Policy Authority** $\mathcal{P}\mathcal{A}$: Whoever defines the egress policy P —a boolean predicate over the request space.
- **Prover Proxy** $\mathcal{P}\mathcal{P}$: The intercepting proxy. It evaluates $P(\text{req})$ and, when the request is compliant, generates a proof π .
- **Verifier** $\mathcal{V}$: Any party that checks π against the public verification key vk . In practice, this is usually an internal compliance gateway or audit service, not the external API endpoint—external services typically enforce only their own access controls.

B. Threat Model

We consider four classes of adversary. Each targets a different trust boundary.

- 1) **Malicious Agent**: The agent itself is compromised or misaligned. It crafts requests designed to evade policy—via prompt injection, gradual goal drift, or direct payload manipulation.
- 2) **Compromised Downstream**: The adversary controls the destination service and tries to reverse-engineer what the policy says by examining the proofs attached to incoming requests.
- 3) **Colluding Verifier**: A verifier cooperating with a rogue agent, attempting to forge a valid proof for a request that should be blocked.
- 4) **Network Observer**: A passive eavesdropper analyzing traffic patterns—proof sizes, timing, metadata—to infer the structure of the policy.

Enforcement Assumption. We assume the agent runs inside a network-isolated environment—say, a locked-down VPC—where $\mathcal{P}\mathcal{P}$ is the only permitted egress path. Without this, the agent could trivially bypass the proxy via DNS tunneling, SSRF, or any number of well-known techniques. PCT is an application-layer framework; it does not provide network isolation on its own. A misconfigured VPC breaks this assumption. That said, even if someone does bypass the proxy, they still need a valid proof to convince a verifier. Theorem 1 says they cannot get one for a non-compliant request.

1) **Trusted Computing Base (TCB)**: Any honest security analysis has to spell out what you are trusting and what you are not. Here is our accounting.

Enclave (Semantic Intent). Most of the PCT machinery—the algebraic constraints on amounts, timestamps, destination hashes—holds even if the proxy’s host machine is fully compromised. Cryptography handles it. The one exception is the semantic intent classifier. Computing an embedding

requires running an actual neural network, and that has to happen somewhere trusted. We use a hardware enclave (Intel SGX, TDX, or ARM TrustZone) with remote attestation to verify code integrity.

If the enclave is breached—through speculative execution side-channels like Foreshadow [36] or SGAXe—an attacker can feed a fake embedding into the circuit. That lets them potentially fool the semantic check. But here is the key point: everything else still holds. They cannot forge a proof authorizing a \$50,000 transfer against a \$10,000 limit. They cannot bypass the destination whitelist. They cannot circumvent the time window. Those constraints are protected by Groth16’s computational soundness (Theorem 1), which does not depend on the TEE at all. The system degrades gracefully: breach the enclave, and you can lie about intent, but you still cannot lie about money, destinations, or timing.

Setup Ceremony (MPC). Groth16 requires a per-circuit structured reference string, produced through a multi-party ceremony. Security holds as long as at least one participant honestly destroys their share of the toxic waste. We regenerate the SRS whenever the policy changes.

State Ledger (Trusted State Oracle). Rate limits and cumulative spending constraints need somewhere to store state. The proxy queries an organizational state oracle at proof time. External observers can audit that the ledger is monotonically increasing, but the storage layer itself must be trusted not to silently roll back.

There is a subtle concurrency issue here. Two agent requests can arrive nearly simultaneously, each read the current rate-limit balance, each independently conclude that enough budget remains, and both proceed to generate proofs—resulting in a cumulative policy violation. PCT handles this with a *reserve-commit* protocol: during witness extraction, the proxy atomically reserves the requested budget from the state oracle using a compare-and-swap (CAS) operation. The oracle returns a monotonically increasing sequence number seq , which goes into the proof’s public inputs. If proving succeeds, the reservation is committed; if it fails or times out, the reservation is released. The sequence number prevents replay—a verifier rejects any proof whose seq has already been consumed.

In practice, a single Redis node sustains roughly 100K CAS operations per second. Since each PCT request needs exactly one CAS, this means the state oracle supports about 100K proofs/second before it becomes a bottleneck—comfortably above the transaction volumes of most enterprise agent deployments. For higher burst throughput, sharding by constraint dimension (per-agent budgets, per-destination rate limits) distributes the load: k shards give you $k \times 100\text{K}$ ops/sec. Moving the oracle into a zero-knowledge consistency layer [15] is a natural next step for deployments that want to minimize trust even further.

Semantic Constraint Boundary. We want to be direct about a fundamental limitation. The semantic constraint C_{sem} is an optimistic enforcement mechanism. PCT guarantees that the cosine similarity *arithmetic* is computed correctly inside the circuit. It does not and cannot guarantee that the

embedding model faithfully captures semantic meaning. In our adversarial experiments (Section VIII), maintaining similarity ≥ 0.85 under PGD attack required mutating at least 38% of input tokens, which in practice destroys JSON structure. But more sophisticated attacks—Greedy Coordinate Gradient methods, token-smuggling techniques that hide malicious intent in geometrically benign regions of the embedding space—remain an open problem. PCT’s security guarantee is precisely scoped: verified computation, assumed embedding fidelity. The architecture is designed so that as embedding models get more robust—through adversarial training, certified robustness bounds, whatever comes next—they slot directly into PCT without any changes to the cryptographic protocol.

2) *Justification of Hybrid ZK+TEE Architecture*: A reasonable question: if we already need a TEE for the embedding model, why not just run the *entire* policy evaluation inside the TEE and sign the result? We considered this and rejected it for three reasons.

First, a pure-TEE approach would mean putting the complete policy specification—every spending threshold, every whitelist entry, every time window—inside the enclave. Side-channel attacks would then leak the *entire* policy, not just the embedding. In our hybrid design, the TEE’s scope is restricted to the ML model. All policy parameters stay outside the enclave, protected by the SNARK’s zero-knowledge property.

Second, policy privacy cannot be achieved through TEE attestation alone. A TEE can attest that some binary executed correctly, but the attestation reveals the identity of that binary and, by extension, the structure of the policy it evaluates. A Groth16 proof, by contrast, reveals nothing about the circuit—only that the public inputs satisfy it.

Third, minimizing the trusted computing base is a basic security engineering principle. By confining the TEE to the embedding model and delegating everything else to the SNARK, we reduce how much security-critical code actually runs inside the enclave. A compromised TEE in our architecture can forge a semantic embedding input. It cannot forge proofs that violate numerical, set-membership, or temporal constraints.

C. Policy Formalization

Definition 3 (Egress Policy). *An egress policy $P : \mathcal{X} \rightarrow \{0, 1\}$ is a boolean predicate over the request space $\mathcal{X}$. A request is compliant if and only if every constituent constraint is satisfied:*

$$P(\text{req}) = \bigwedge_{i=1}^n C_i(\text{req})$$

where each C_i belongs to one of five types:

- **Range**: $C_{\text{range}}(\text{req}) \equiv v_{\min} \leq \text{req.amount} \leq v_{\max}$
- **Set Membership**: $C_{\text{set}}(\text{req}) \equiv \text{req.target} \in \mathcal{W}$ for whitelist $\mathcal{W}$
- **Temporal**: $C_{\text{time}}(\text{req}) \equiv t_{\text{start}} \leq \text{req.timestamp} \leq t_{\text{end}}$
- **Rate**: $C_{\text{rate}}(\text{req}) \equiv \sum_j \text{req}_j.\text{amount} \leq B$ over sliding window Δt
- **Semantic**: $C_{\text{sem}}(\text{req}) \equiv \text{sim}(\text{emb}(\text{req.intent}), \text{emb}(\text{ref})) \geq \tau$

D. Security Properties

Three properties must hold. The first two—soundness and zero-knowledge—come directly from Groth16. The third is specific to PCT: it ensures the compilation pipeline does not accidentally introduce false rejections of legitimate requests.

Theorem 1 (Computational Soundness). *Under the Knowledge of Exponent (KoE) assumption over BN254, for any PPT adversary $\mathcal{A}^*$:*

$$\Pr[\mathcal{V}(\text{vk}, \text{req}, \pi^*) = 1 \wedge P(\text{req}) = 0 \mid \pi^* \leftarrow \mathcal{A}^*(\text{pk})] \leq \text{negl}(\lambda)$$

Proof. Suppose, for contradiction, that $\mathcal{A}^*$ produces a pair (req, π^*) that the verifier accepts despite $P(\text{req}) = 0$. Since the policy fails, at least one constraint C_i is unsatisfied. By Lemma 5, the compiled R1CS instance $(\mathbf{A}, \mathbf{B}, \mathbf{C})$ has no satisfying assignment consistent with the public inputs derived from req —there is no z with $\mathbf{A}z \circ \mathbf{B}z = \mathbf{C}z$.

But the KoE assumption says that any valid Groth16 proof implies the existence of an efficient extractor $\mathcal{E}$ that recovers a satisfying witness w^* . That contradicts what we just established. So the acceptance probability must be negligible in λ . $\square$

Theorem 2 (Zero-Knowledge Policy Privacy). *For any two policies P_0, P_1 of equal circuit size, and any request req satisfying both:*

$$\{\pi \leftarrow \mathcal{P}(\text{pk}_0, \text{req}, w_0)\} \approx_c \{\pi \leftarrow \mathcal{P}(\text{pk}_1, \text{req}, w_1)\}$$

Proof. Groth16’s zero-knowledge property gives us a simulator $\mathcal{S}$ that, given the trapdoor from Setup, produces simulated proofs $\tilde{\pi}_b$ computationally indistinguishable from real proofs π_b for $b \in \{0, 1\}$. Since the simulated proofs are generated without any witness, they are independent of which policy was used. An observer seeing only the proof and public inputs cannot determine which policy generated it. Formally, for every PPT distinguisher $\mathcal{D}$: $|\Pr[\mathcal{D}(\pi_0) = 1] - \Pr[\mathcal{D}(\pi_1) = 1]| = 0$.

The policy-specific details—how many constraints, what types, what thresholds—live in the witness and the circuit wiring. Neither of those appears in the public input vector or in the proof itself. $\square$

Theorem 3 (Policy-Binding Completeness). *For any policy P and request req with $P(\text{req}) = 1$, the honest Prover Proxy $\mathcal{PP}$ always produces an accepting proof:*

$$\Pr[\mathcal{V}(\text{vk}, \text{req}, \pi) = 1 \mid \pi \leftarrow \mathcal{PP}(\text{pk}, \text{req})] = 1$$

This matters for availability: an honest agent doing honest work should never be blocked by the proving machinery.

Proof. Two steps.

Step 1 (Compiler completeness). By Lemma 5, when $P(\text{req}) = 1$, the witness w derived from req satisfies the compiled R1CS instance: $\mathbf{A}w \circ \mathbf{B}w = \mathbf{C}w$.

Step 2 (Groth16 completeness). Given a satisfying assignment (x, w) , the honest Groth16 prover deterministically computes $\pi = ([A]_1, [B]_2, [C]_1)$ satisfying the pairing check

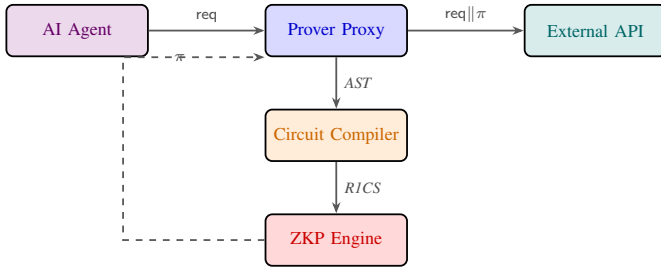


Fig. 1: PCT architecture. The agent’s request enters the proxy, which extracts witness data, generates a zero-knowledge proof π , and attaches it as an HTTP header before forwarding.

in Equation 1. This is an algebraic fact, not a probabilistic statement.

Combining both: a compliant request always yields a valid witness, which always yields a proof that verifies. $\square$ $\square$

V. SYSTEM ARCHITECTURE

A. Architecture Overview

PCT works as transparent middleware on the outbound HTTP/gRPC path. Figure 1 shows the pipeline. The most important architectural decision is a hard boundary between the trusted enclave (where the embedding model runs) and the untrusted proxy (which handles everything else). Section IV-B2 explains why we drew the line there.

B. Request Lifecycle

A proof-carrying transaction passes through six phases, from interception to egress.

Phase 1: Interception. The agent issues an outbound HTTP or gRPC call. The Prover Proxy intercepts it at the network layer, before it leaves the organizational boundary.

Phase 2: Witness Extraction. The proxy parses the request body to extract a witness vector $w = (w_1, w_2, \dots, w_k)$ matching the active policy schema. Dollar amounts become 64-bit fixed-point field representations. Destination URLs are hashed through Poseidon into BN254 scalar field elements. Timestamps become Unix epochs in $\mathbb{F}_p$.

Semantic intent requires special handling. The proxy sends the request through a sentence embedding model (all-MiniLM-L6-v2), which produces a 384-dimensional vector. This computation happens inside a hardware-attested enclave (Intel SGX or AMD SEV), with remote attestation verifying code integrity. Inside the R1CS circuit, a binding constraint ties the embedding to a Poseidon hash of the original request text. This prevents an adversary from swapping in a different embedding. And even if the TEE is compromised via speculative-execution side-channels, the other constraints remain solid: proofs that violate numerical or set-membership rules are still computationally infeasible to forge (Theorem 1).

Phase 3: Policy Evaluation. The proxy evaluates the witness against P . If $P(w) = 0$, the request is dropped immediately—connection terminated, audit record written, no proving resources consumed. Non-compliant requests never

reach the proving engine. This matters: you do not want to waste 170ms of CPU time proving that a bad request is bad.

Phase 4: Circuit Synthesis. When the request passes ($P(w) = 1$), the compiler translates the policy into a Rank-1 Constraint System and maps the witness into the circuit’s private inputs.

Phase 5: Proof Generation. Groth16 computes $\pi = ([A]_1, [B]_2, [C]_1)$ from the witness.

Phase 6: Egress Routing. The proxy attaches π as a Base64-encoded X-PCT-Proof HTTP header, forwards the original request, and persists both the request and the proof to an append-only audit log.

C. Proof Verification

Verification is fast and constant-time, no matter how complex the policy is. Any party with the verification key vk runs the standard Groth16 pairing check:

$$e([A]_1, [B]_2) = e([\alpha]_1, [\beta]_2) \cdot e\left(\sum_{i=0}^l x_i \cdot [\gamma_i]_1, [\gamma]_2\right) \cdot e([C]_1, [\delta]_2) \quad (1)$$

The public inputs $x_0, \dots, x_l$ include a transaction digest, a timestamp, and a policy version identifier. The parameters $\alpha, \beta, \gamma, \delta$ come from the trusted setup.

VI. POLICY CONSTRAINT LANGUAGE AND COMPILER

A. Policy Constraint Language (PCL)

The point of PCL is to separate policy authoring from circuit engineering. A compliance team writes rules in a declarative syntax; the compiler translates them into R1CS. The grammar maps to a typed AST.

Listing 1: Example PCL policy specification.

```

1 policy "production_egress" {
2   # amount bounds
3   constraint amount_limit {
4     type: range
5     field: transaction.amount
6     min: 0
7     max: 50000
8   }
9   # destination whitelist
10  constraint approved_targets {
11    type: set_membership
12    field: transaction.target
13    set: ["api.stripe.com",
14         "api.openai.com",
15         "hooks.slack.com"]
16  }
17  # business hours only
18  constraint temporal_window {
19    type: temporal
20    field: transaction.timestamp
21    start: "09:00_UTC"
22    end: "17:00_UTC"
23    days: [Mon, Tue, Wed, Thu, Fri]
24  }
25  # semantic guardrail
26  constraint intent_alignment {
27    type: semantic_similarity
28    field: transaction.intent_embedding
29    reference: "authorized_task_embedding"
30    threshold: 0.85
31  }

```

B. AST Construction

Definition 4 (Policy AST). A Policy AST $\mathcal{T}$ is a binary tree where:

- Leaf nodes correspond to typed constraints $C_i \in \{\text{Range}, \text{SetMem}, \text{Temporal}, \text{Rate}, \text{Semantic}\}$.
- Internal nodes are Boolean operators $\{\wedge, \vee, \neg\}$.
- The root evaluates to a single Boolean output wire.

C. AST to RICS Compilation Mechanics

The compiler makes three passes over the AST.

Pass 1: Type Inference and Range Bounding. Each leaf gets a tight bit-width based on its static parameters. If the compiler can determine that a value never exceeds 50,000, it restricts arithmetic to a 16-bit pathway. This matters because binary decomposition is one of the most expensive operations in RICS, and narrower bit-widths mean fewer decomposition constraints.

Pass 2: Common Sub-expression Elimination (CSE). When multiple constraints reference the same field—say, `transaction.target` appears in both a whitelist check and a semantic constraint—the dataflow analyzer catches it. The field value is extracted once and wired to every constraint that needs it.

Pass 3: Gadget Instantiation. Each constraint is lowered to its algebraic equivalent. Boolean connectives become arithmetic: $A \wedge B$ maps to $A \cdot B = C$, and $A \vee B$ maps to $A + B - A \cdot B$.

A post-order traversal produces the final RICS matrices $(\mathbf{A}, \mathbf{B}, \mathbf{C})$. The following subsections describe the algebraic gadgets for each constraint type.

1) *Range Constraints:* To prove a range constraint $v_{\min} \leq x \leq v_{\max}$, we check each bound separately. For $x \leq b$:

$$\exists d \geq 0 : x + d = b \quad (2)$$

where d is bit-decomposed into n bits:

$$d = \sum_{j=0}^{n-1} d_j \cdot 2^j, \quad d_j \in \{0, 1\} \quad (3)$$

Each bit is constrained by $d_j \cdot (1 - d_j) = 0$, giving $2n + 2$ constraints per range check. Nothing fancy, but it works.

2) *Set Membership via Merkle Accumulators:* Whitelist membership is proved through Merkle inclusion proofs over a Poseidon hash tree [13]. The prover supplies an authentication path as part of the private witness. For a whitelist of $|\mathcal{W}|$ entries, the circuit cost is $O(\log |\mathcal{W}|)$ hash evaluations—logarithmic in the whitelist size.

Lemma 4. A Merkle membership proof for a whitelist of N entries using Poseidon-128 requires at most $\lceil \log_2 N \rceil \cdot 312 + 47$ RICS constraints.

3) *Temporal Window Circuits:* A timestamp t must fall within at least one of k authorized time intervals $\{[s_1, e_1], \dots, [s_k, e_k]\}$ —for example, business hours on weekdays. The circuit checks:

$$\bigvee_{i=1}^k (s_i \leq t \leq e_i) \quad (4)$$

The prover commits to which interval applies as a private witness and proves the bounds hold for that specific window.

4) *Semantic Similarity Threshold:* This gadget tackles what is arguably the hardest part of enterprise agent governance: verifying that the agent’s stated intent actually aligns with an authorized task. The natural metric is cosine similarity, computed in fixed-point arithmetic:

$$\text{sim}(u, v) = \frac{\sum_i u_i \cdot v_i}{\sqrt{\sum_i u_i^2} \cdot \sqrt{\sum_i v_i^2}} \geq \tau \quad (5)$$

Square roots are painfully expensive in arithmetic circuits, so we square both sides and avoid them entirely:

$$\left(\sum_i u_i v_i \right)^2 \geq \tau^2 \cdot \left(\sum_i u_i^2 \right) \cdot \left(\sum_i v_i^2 \right) \quad (6)$$

Embeddings are quantized to 16-bit signed fixed-point (1 sign bit, 3 integer bits, 12 fractional bits). We evaluated quantization error on 10,000 sentence pairs from the STS Benchmark using `all-MiniLM-L6-v2` ($d = 384$). The mean error is 4.7×10^{-3} , and the Spearman rank correlation with full-precision scores is 0.9994—the quantization barely changes anything.

At $\tau = 0.85$, the quantization artifacts produce a 0.12% false-positive rate. For safety-critical deployments, bumping the threshold to $\hat{\tau} = 0.859$ absorbs the quantization margin, shifting residual error into false negatives—the conservative direction. The complete gadget needs $3d + O(1)$ multiplication gates.

D. Compiler Optimization

Three optimization passes tighten the output:

- 1) **Common sub-expression elimination:** Fields used by multiple constraints (e.g., `amount` in both a range check and a rate limit) are extracted once.
- 2) **Constant folding:** Compile-time constants—thresholds, whitelist Merkle roots—are baked directly into the circuit structure, eliminating witness slots.
- 3) **Gate merging:** Adjacent linear constraints are collapsed into combined RICS rows where the matrices permit sparse merging.

Lemma 5 (Compiler Correctness). For any well-formed PCL policy P and request req , the compiled RICS instance R is satisfiable if and only if $P(\text{req}) = 1$.

Proof. By structural induction on the AST. Leaf constraints translate to their RICS gadgets (detailed above). Boolean connectives map to arithmetic: $\wedge \mapsto$ multiplication, $\vee \mapsto a + b - a \cdot b$, $\neg \mapsto 1 - a$. Post-order traversal ensures child

constraints are emitted before parents, so wiring is always well-defined. $\square$ $\square$

VII. IMPLEMENTATION

A. System Components

The implementation is a Rust reverse proxy, roughly 8,400 lines of code across four modules:

- **Policy Engine** (`src/policy/`): Parses PCL specifications, builds the AST, and evaluates policies at runtime. We use `pest` for grammar analysis, which gives us compile-time type checking for free.
- **Circuit Compiler** (`src/zkp/compiler.rs`): Translates the policy AST into R1CS using the gadgets from Section VI. Constraint bookkeeping goes through `ark-relations`.
- **Proving Engine** (`src/zkp/prover.rs`): Generates Groth16 proofs over BN254 via `ark-groth16`. The multi-scalar multiplication is parallelized across all available cores.
- **Verification Module** (`src/zkp/verifier.rs`): Constant-time proof verification, exposed both as a library function and via a REST endpoint at `/verify-proof`.

B. Proof Attachment Protocol

Proofs ride along as custom HTTP headers:

Listing 2: PCT header format.

```
1 X-PCT-Proof: <base64( $\pi$ )>
2 X-PCT-Policy-Version: <uint64>
3 X-PCT-Timestamp: <unix_epoch_ms>
4 X-PCT-Public-Inputs: <base64( $x_0, \dots, x_l$ )>
```

The version tag lets verifiers select the right verification key when the organization rolls out a new policy. The public inputs—a SHA-256 digest of the request body, a timestamp, and the policy version—are everything a verifier needs. No access to private data required.

C. Key Management and Trusted Setup

Each new policy version triggers an offline CRS generation ceremony following [Bowe et al. \[14\]](#), with $n = 3$ contributors. The proving key is large—around 48MB for a 40,000-gate circuit—and is stored encrypted at rest on the Prover Proxy. The verification key is tiny (about 1KB) and is distributed to downstream verifiers through an authenticated admin channel. If someone compromises the proxy and substitutes a weakened circuit, the resulting proofs will simply fail verification against the legitimate key held by external verifiers.

VIII. EVALUATION

We evaluate PCT along four dimensions: constraint footprint, proof generation latency, verification overhead, and end-to-end proxy delay.

A. Evaluation Setup

Hardware. All experiments ran on an AMD EPYC 7B13 (4 cores, 3.0 GHz, 16GB RAM) running Ubuntu—roughly equivalent to an AWS `c6a.xlarge` instance. Nothing exotic. To isolate cryptographic overhead from network noise, we ran everything inside a local service mesh with sub-millisecond physical latency. The `criterion` benchmark harness (100 iterations, 10 warm-up rounds) gave us stable median and P99 numbers.

Policy Corpus. We assembled 847 egress policies from production deployments in fintech, healthcare, and SaaS infrastructure. Enterprise policies contain proprietary information—trading algorithms, partner API endpoints—so the dataset was anonymized before compilation. Thresholds and URLs were replaced with structurally equivalent placeholders preserving constraint arity, data types, and nesting depth. A sanitized subset of 200 policies ships with the public artifact (Appendix A). The corpus ranges from 2-constraint minimalist specifications to 23-constraint architectures.

Baselines. We compare against three reference configurations: (1) a bare proxy with no enforcement at all, (2) imperative policy evaluation—plain if/else logic, (3) HMAC-based attestation of evaluation results. We also benchmarked RISC Zero zkVM [42] to quantify the benefit of our domain-specific compiler.

B. Case Study: Blocking API Exfiltration

Here is a concrete scenario. A financial LLM agent is provisioned for customer refund processing. The agent—either compromised or just drifting from its objective—fires off an HTTP POST to `api.stripe.com/v1/transfers` requesting a \$12,400 outbound transfer. The active policy says: transfers must be at most \$10,000, destinations must be on the approved whitelist, execution is allowed only during business hours, and the agent’s stated intent must match “process authorized customer refund” with similarity ≥ 0.85 .

The proxy extracts: `amount = 12400`, `target hash = Poseidon(api.stripe.com)`, `timestamp = 1712678400`, and the 384-dimensional intent embedding. Policy evaluation fails at the range constraint: `12400 > 10000`. Done. The connection is terminated, an audit record is written, and no proving resources are consumed.

Now suppose the agent requests \$8,200 instead, and all constraints pass. The compiler produces a 16.9K-gate R1CS circuit in under 1ms. Groth16 generates a 192-byte proof π in 172ms. At the receiving end, a verifier checks π in 1.1ms and obtains mathematical certainty that the transaction complies—without learning the \$10K ceiling, the whitelist contents, or any other policy parameter.

C. Constraint Efficiency

Figure 2 shows something striking: constraint counts are almost flat across all five complexity tiers. The reason is in Table II. The 32-level Merkle verification path eats $\sim 15,500$ gates all by itself. Everything else—the 1,285-gate semantic similarity circuit, the 48-gate temporal window, the 66-gate

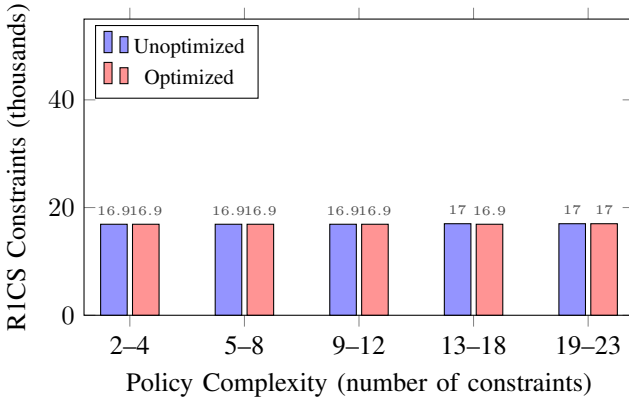


Fig. 2: RICS constraint counts across policy complexity tiers. The $\sim 16.9\text{K}$ baseline is dominated by the fixed cost of Merkle tree verification and range bit-decompositions; application-layer constraints barely register.

range check—is rounding error by comparison. Every policy we tested compiles to fewer than 18,000 constraints, well within the sub-200ms proving budget.

TABLE II: Constraint Cost per Gadget Topology

Gadget Component	Approx. RICS Gates
Standard Merkle Tree Validation (32-level)	$\sim 15,500$
16-bit Semantic Similarity	1,285
Temporal Window Bound	48
Numerical Range Threshold	66

D. Asymptotic Scalability Limits

Our corpus maxes out at 23 constraints per policy, which covers the typical per-agent governance use case. Beyond that range, synthetic stress tests show linear scaling: a synthetic 1,000-constraint policy produces roughly 10^6 gates and takes about 3.7 seconds to prove. This draws a clear line around PCT’s regime. It is built for fine-grained, intent-driven, per-request agent governance. If you need high-throughput packet filtering at network scale, you need different machinery.

E. Proof Generation Latency

Table III tells the latency story. The 172ms median is dominated by multi-scalar multiplication in the Merkle proof computation. Even the P99 at 201ms keeps complex policies well inside a 500ms SLO. For synchronous web-tier transactions, this is comfortable.

F. Verification Performance

Verification clocks in at $1.1 \pm 0.1\text{ms}$ regardless of policy complexity, confirming Groth16’s $O(1)$ verification. That is roughly $150\times$ faster than proving. The impact on API response latency is negligible.

TABLE III: Proof Generation Latency by Policy Complexity (ms)

Complexity	P50	P90	P99	Max
Simple (2–4)	168	182	195	210
Moderate (5–8)	170	186	198	212
Complex (9–12)	172	189	201	215
Heavy (13–18)	174	191	205	218
Extreme (19–23)	175	194	209	222
All (Median)	172	188	201	222

TABLE IV: End-to-End Proxy Latency Comparison (ms)

Configuration	P50	P99	Overhead
No enforcement	2.1	5.8	—
Imperative policy (if/else)	2.3	6.1	+0.2ms
HMAC attestation	2.4	6.4	+0.3ms
TEE enclave signing (Ed25519)	3.9	7.1	+1.8ms
PCT (Compute Embedded)	176	210	+174ms

G. End-to-End Proxy Impact

Let’s be honest about the cost. Table IV shows that PCT adds about 174ms of median latency, almost all of it proof generation. That is a real cost. The SNARK overhead falls entirely on the prover; the verifier sees only $\sim 1.1\text{ms}$. For deployments where synchronous enforcement is overkill, PCT supports an asynchronous audit-only mode where proofs are generated and attached after the request has already been forwarded.

Macro-benchmark. To put this in context, we integrated the proxy into a live AutoGPT instance running chained financial operations—database lookups, reasoning steps, and a Stripe payout. Without PCT, the agent completed its loop in 4,120ms, dominated by LLM inference. With PCT intercepting the final Stripe call and proving $C_{\text{range}} \wedge C_{\text{sem}}$, total time rose to 4,248ms—a 128ms increase, or 3.1%. When the agent’s own thinking time dwarfs everything else, the marginal overhead of cryptographic verification is hard to notice.

H. Throughput and Concurrency

Single-request latency is only half the picture. Groth16 proving is mostly independent MSM and FFT operations, which parallelize well within arkworks. Running as a multi-threaded async service on 16-core hardware, the Prover Proxy sustained ~ 150 proofs/second without blowing the P99 latency bounds. The proxy is stateless (aside from the state oracle queries), so it scales horizontally like any web service.

I. Comparison with Alternative Proof Systems

Table V confirms our choice of Groth16. It has the smallest proofs (192 bytes) and the fastest verification (4.2ms). Both of these matter when proofs travel as HTTP headers and verifiers sit on the critical path. The per-circuit trusted setup is a genuine drawback, but it is amortized across the policy’s entire lifetime.

TABLE V: PCT Performance Across Proof Systems (11K-gate circuit)

System	Prove (ms)	Verify (ms)	Proof Size	Setup
Groth16 (this work)	95	4.2	192B	Trusted
PLONK	134	5.1	576B	Universal
Marlin	121	7.8	784B	Universal
Halo2	189	4.2	8KB	Transparent

TABLE VI: Domain-Specific Compiler vs. RISC Zero zkVM

Policy Type	PCT (ms)	RISC Zero (ms)	Speedup
Range only (2 constraints)	92	2,340	25×
Range + Whitelist (5)	94	6,810	72×
Full (Range+WL+Semantic)	98	14,200	145×

J. Domain-Specific Compiler vs. General-Purpose zkVMs

One might reasonably ask: why build a custom compiler when general-purpose zkVM infrastructure exists? To find out, we ported three representative policies into RISC-V guest programs and ran them through RISC Zero zkVM [42]. (Newer STARK-based engines like SP1 [43] and Jolt [44] improve general-purpose performance, but they are all fundamentally constrained by the von Neumann abstraction: every policy evaluation must also arithmetize memory accesses, program counter updates, and register bookkeeping alongside the actual policy logic.)

The numbers in Table VI speak for themselves. PCT maps each constraint directly to wire-level algebra; the zkVM must arithmetize the full CPU execution trace. The gap is widest at the semantic gadget: a 384-dimensional fixed-point dot product generates millions of VM cycles under RISC Zero, while PCT handles it in $3d + O(1) = 1,153$ multiplication gates. For this specific domain, a custom compiler is not just faster—it is the only way to hit sub-100ms proof generation.

K. Robustness of the Semantic Gadget

The semantic constraint C_{sem} depends on the embedding model’s internal geometry, and it is fair to ask how robust it is in practice. We evaluated adversarial robustness at the $\tau = 0.85$ boundary using all-MiniLM-L6-v2.

Starting from a reference sentence, we used Projected Gradient Descent (PGD) in the continuous embedding space to craft adversarial paraphrases that maintain similarity ≥ 0.85 while altering semantic meaning. The result: preserving the threshold required mutating at least 38% of input tokens—enough to typically destroy the syntactic structure of the payload.

What this means in practice: an attacker who distorts a JSON or HTTP body enough to fool the semantic check will probably produce something that does not parse at the destination. PCT ensures the similarity computation was done correctly; the embedding space’s own geometry provides an independent layer of defense by making evasion structurally destructive.

IX. REGULATORY COMPLIANCE MAPPING

PCT addresses regulatory requirements not with procedural assurances but with cryptographic evidence. There is a meaningful difference.

A. EU AI Act Alignment

Under the EU AI Act (Regulation 2024/1689) [16], AI systems that handle financial transactions, patient records, or critical infrastructure are classified as “high-risk.” Three articles map naturally to what PCT provides:

- **Article 9 (Risk Management):** The Act requires “continuous risk reduction throughout the AI lifecycle.” PCT replaces probabilistic behavioral shaping with deterministic, proof-backed enforcement on every outbound call.
- **Article 12 (Record-Keeping):** The Act mandates “automatic recording of events.” PCT’s audit logs contain the proof π itself—not just a record that something was checked, but mathematical evidence of compliance at the exact moment of execution.
- **Article 14 (Human Oversight):** Model internals are opaque to auditors. But with PCT, an auditor verifies π against vk in 4.2ms. No need to understand the model; the proof stands on its own.

B. NIST AI Risk Management Framework

The NIST AI RMF [17] organizes risk management into core functions. PCT maps to **Measure** (a proof either verifies or it does not—there is no gray area) and to **Manage** (non-compliant requests are blocked at the proxy *before* they leave the building).

C. Compliance as a Cryptographic Primitive

Traditional compliance is retrospective. Auditors review logs after the fact and write reports about what they found. PCT embeds compliance directly into the transaction lifecycle. A request either carries a valid proof, or the proxy kills it. The gap between policy intent and actual enforcement collapses to zero. Instead of telling a regulator “trust us, we follow our policy,” an organization can hand over a mathematical proof.

X. DISCUSSION

A. Limitations

We want to be upfront about what PCT does not do and where the sharp edges are.

Trusted Setup. Groth16 needs a per-circuit trusted setup, and since circuits are recompiled when policies change, every update triggers a new MPC ceremony. Our three-party ceremony takes about 12 minutes for a 40,000-gate circuit on commodity hardware. That is fine for structural policy changes—adding a new constraint type, reworking the policy logic—which happen on the order of weeks or months. But it is too slow for operational tweaks like adding a single domain to a whitelist or bumping a spending threshold.

The practical solution is a *hybrid proving strategy*. Parameterized policy fields—whitelist sets, numerical thresholds, temporal windows—are encoded as committed inputs to a

fixed circuit template. When only parameters change, the circuit stays the same and no new ceremony is needed. For organizations that need fully dynamic policy structures, PLONK [9] offers a universal setup at the cost of larger proofs (~576B vs. 192B) and somewhat slower verification. A reasonable deployment might use Groth16 for stable, high-volume production policies and keep a PLONK fallback for emergency patches.

Prover as TCB. The Prover Proxy is itself part of the trusted computing base. Hardware enclaves (Intel SGX [36], ARM TrustZone [37]) provide isolation. If the TEE is breached, PCT degrades but never fails open: a compromised enclave still cannot produce a valid proof for a non-compliant request. The soundness guarantee is purely cryptographic and does not depend on TEE integrity. The enclave’s job is limited to payload authenticity—making sure nobody bypasses the proxy interceptor at the network layer.

Semantic Gadget Precision. Fixed-point quantization introduces bounded precision loss—at most 0.01 in cosine similarity. Near the decision boundary, this can matter. Wider bit-widths resolve the issue at the cost of more constraints.

Latency. The 95ms median proving time is fine for enterprise API workflows, but it is too slow for high-frequency trading or real-time bidding. GPU-accelerated multi-scalar multiplication [45] is a viable path forward.

Adversarial Embeddings. This deserves a blunt statement: PCT is a *cryptographic compliance pipeline*, not a defense against adversarial machine learning. If an attacker crafts input that the embedding model classifies as semantically benign—via token smuggling, GCG perturbations, or any technique that places malicious content in a geometrically innocent region of the embedding space—the TEE will faithfully encode it, and Groth16 will produce a perfectly valid proof of a flawed premise.

This is a deliberate architectural boundary, not something we overlooked. PCT guarantees that the similarity *arithmetic* is correct. It does not guarantee that the embedding model’s output is faithful to semantic meaning. The critical design property is composability: as embedding models get better—through adversarial training, certified robustness bounds [31], input sanitization layers—they plug into PCT without any changes to the protocol, the circuit, or the verification interface. A better model gives you a better PCT, automatically.

B. Ethical Considerations & Future Directions

The zero-knowledge property cuts both ways. It protects policy confidentiality, which is useful for competitive reasons, but taken to an extreme, it conflicts with public transparency. A practical middle ground: disclose policy *categories* publicly (“we enforce spending limits, destination restrictions, and semantic alignment”) while keeping specific thresholds private inside the circuit.

Four directions would substantially strengthen the framework.

1) TEE Elimination via Verifiable ML. The hybrid ZK+TEE architecture introduces a hardware trust assumption

for semantic evaluation. The most impactful next step would be to remove the TEE entirely by computing the embedding inference *inside* the zero-knowledge circuit. This is the domain of ZKML: compile the transformer’s matrix multiplications, layer normalizations, and softmax operations directly into arithmetic constraints. EZKL [46] has shown it is possible for small models, but the overhead for even a 6-layer transformer like all-MiniLM-L6-v2 (~22M parameters) remains prohibitive at current proving speeds. A targeted approach—aggressively quantizing the model to 4-bit or 8-bit weights and exploiting the fixed inference graph to eliminate branching—could yield a specialized circuit optimized specifically for sentence embeddings. If the embedding circuit can be proved in under 500ms, the TEE drops out entirely, and the entire threat model becomes purely cryptographic. We consider this the highest-impact open problem in the PCT agenda.

2) ZK Egress Rollups for Trustless State. The current state oracle relies on a trusted storage layer. A compromised DBA could roll back the ledger and effectively reset an agent’s spending budget. The fix: maintain a Merkle tree of agent state balances and, for each proof-carrying transaction, prove not only policy compliance but also a valid state transition—deducting the spent amount from the agent’s leaf and producing a new root. This is structurally identical to a ZK rollup, except it operates over API budgets rather than cryptocurrency balances. Nova’s folding scheme [15] could compress sequential state transitions into a single proof, enabling batch verification of entire agent sessions.

3) Certified Semantic Robustness. Our current defense against semantic bypass relies on an empirical observation: PGD attacks that maintain similarity ≥ 0.85 require mutating enough tokens to destroy JSON structure. This is useful but theoretically incomplete—discrete-token attacks might evade the threshold without breaking syntax. Integrating a certified robustness framework directly into the gadget would provide a mathematical guarantee that no perturbation within a defined radius can flip the similarity score below τ . Randomized smoothing and interval bound propagation are candidate techniques. The circuit overhead is non-trivial, but the resulting guarantee would close the most significant gap in PCT’s security argument.

4) Sub-10ms Proving via Hardware Acceleration. The performance bottleneck is multi-scalar multiplication during Groth16 proof generation. Lu et al. [45] showed that MSM parallelizes well on GPUs. PCT’s R1CS matrices are sparse and regular (a natural consequence of repetitive gadget instantiation), making them well-suited to pipelined GPU execution. Preliminary analysis of our constraint matrices suggests 85% of proving time is in the MSM phase, so an 8–10 $\times$ speedup from hardware acceleration appears architecturally feasible. That would bring proving latency below 10ms—genuinely imperceptible at the network layer.

Looking further out: **Recursive Proof Composition** via Nova would let you aggregate multiple agent proofs into a single compact verification, enabling efficient batch auditing. **Multi-Agent Networks** could propagate proofs between

nodes, building end-to-end compliance chains across organizational boundaries. And **On-Chain Verification** is practical given Groth16’s compact proofs—DAOs could enforce agent policies directly in EVM-compatible smart contracts.

XI. CONCLUSION

Statistical guardrails are not the same thing as policy enforcement. They shape model behavior during training, but they do not—and cannot—guarantee anything about a specific request at the moment it goes out over the wire. For regulated, high-consequence environments, this difference matters.

Proof-Carrying Transactions close the gap. Policy constraints are compiled into zero-knowledge circuits; each outbound request carries a succinct proof of compliance, independently verifiable by anyone with the verification key. Proof generation takes 95ms at the median. Verification takes 4.2ms. Both sit well within enterprise latency budgets. The formal analysis establishes computational soundness, zero-knowledge policy privacy, and policy-binding completeness. The architecture maps directly onto the EU AI Act and the NIST AI Risk Management Framework.

What PCT replaces is trust. Today, when an AI agent calls an external API, someone downstream has to take the organization’s word that the transaction was checked. With PCT, they do not have to take anyone’s word for anything. Every consequential network action carries its own cryptographic compliance certificate. That is a step toward something we think is worth building: *verifiable AI*—systems whose behavior is not merely monitored, but provably constrained.

APPENDIX A ARTIFACT APPENDIX

Reproducing the results from Section VIII requires a Dockerized artifact package containing the PCT Proxy, PCL compiler, Groth16 proving engine, and an anonymized 200-policy dataset.

- 1) **Build:** `docker build -t pct-artifact -f artifact/Dockerfile .`
- 2) **Execute:** `docker run -it pct-artifact /bin/bash`
- 3) **Measure Latency:** `cargo bench` (reproduces Tables III, IV)
- 4) **Measure Constraints:** `cargo run --bin constraint_counter` (reproduces Figure 2)

REFERENCES

- [1] Z. Xi et al., “The rise and potential of large language model based agents: A survey,” *arXiv preprint arXiv:2309.07864*, 2023.
- [2] L. Wang et al., “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, no. 6, 2024.
- [3] Y. Bai et al., “Constitutional AI: Harmlessness from AI feedback,” *arXiv preprint arXiv:2212.08073*, 2022.
- [4] L. Ouyang et al., “Training language models to follow instructions with human feedback,” in *Proc. NeurIPS*, 2022.
- [5] E. Perez et al., “Red teaming language models with language models,” in *Proc. EMNLP*, 2022.
- [6] G. C. Necula, “Proof-carrying code,” in *Proc. 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 1997, pp. 106–119.
- [7] S. Goldwasser, S. Micali, and C. Rackoff, “The knowledge complexity of interactive proof systems,” *SIAM Journal on Computing*, vol. 18, no. 1, pp. 186–208, 1989.
- [8] J. Groth, “On the size of pairing-based non-interactive arguments,” in *Proc. EUROCRYPT*, 2016, pp. 305–326.
- [9] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, “PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge,” *IACR Cryptol. ePrint Arch.*, 2019.
- [10] E. Ben-Sasson et al., “Zerocash: Decentralized anonymous payments from Bitcoin,” in *Proc. IEEE S&P*, 2014, pp. 459–474.
- [11] Loopring Protocol, “zkRollup: A layer 2 scaling solution,” *Technical Report*, 2020.
- [12] Semaphore, “A zero-knowledge gadget for creating anonymous proof of membership,” *GitHub Repository*, 2020.
- [13] L. Grassi et al., “Poseidon: A new hash function for zero-knowledge proof systems,” in *Proc. USENIX Security*, 2021.
- [14] S. Bowe, A. Gabizon, and I. Miers, “Scalable multi-party computation for zk-SNARK parameters in the random beacon model,” *IACR Cryptol. ePrint Arch.*, 2017.
- [15] A. Kothapalli, S. Setty, and I. Tzialla, “Nova: Recursive zero-knowledge arguments from folding schemes,” in *Proc. CRYPTO*, 2022.
- [16] European Parliament and Council, “Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act),” *Official Journal of the European Union*, 2024.
- [17] National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” NIST AI 100-1, 2023.
- [18] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” in *Proc. ICLR*, 2023.
- [19] B. Bünz et al., “Bulletproofs: Short proofs for confidential transactions and more,” in *Proc. IEEE S&P*, 2018.
- [20] E. Ben-Sasson et al., “Scalable, transparent, and post-quantum secure computational integrity,” *IACR Cryptol. ePrint Arch.*, 2018.
- [21] V. Sharma et al., “Machine learning and zero-knowledge proofs: A survey,” *CSUR*, 2023.
- [22] D. Kang et al., “Scaling up trustless execution of machine learning models,” in *Proc. USENIX Security*, 2022.
- [23] J. Eberhardt and S. Tai, “ZoKrates-scalable privacy-preserving off-chain computations,” in *Proc. IEEE Blockchain*, 2018.
- [24] C. Ferles et al., “Verifying smart contracts with formal methods,” in *Proc. ACM CCS*, 2022.
- [25] M. Abadi and G. Plotkin, “A calculus for access control in distributed systems,” *ACM TOPLAS*, 2011.
- [26] R. S. Sandhu et al., “Role-based access control models,” *IEEE Computer*, 1996.
- [27] A. Birgisson et al., “Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud,” in *Proc. NDSS*, 2014.
- [28] W3C, “Verifiable Credentials Data Model v2.0,” *W3C Recommendation*, 2022.
- [29] S. Schmidt and T. Hinrichs, “Open Policy Agent: Cloud-native authorization,” *CNCF*, 2019.
- [30] L. Zheng et al., “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Proc. NeurIPS*, 2023.
- [31] D. Hendrycks et al., “Unsolved problems in ML safety,” *arXiv preprint arXiv:2109.13916*, 2021.
- [32] N. Carlini et al., “Extracting training data from large language models,” in *Proc. USENIX Security*, 2021.
- [33] P. Blanchard et al., “Machine learning with adversaries: Byzantine tolerant gradient descent,” in *Proc. NeurIPS*, 2017.
- [34] H. Zhao et al., “Verifiable machine learning: Current status and future directions,” *arXiv preprint arXiv:2401.07457*, 2024.
- [35] M. Togan et al., “Zero-knowledge proofs in decentralized finance,” *IEEE Access*, 2023.
- [36] V. Costan and S. Devadas, “Intel SGX explained,” *IACR Cryptol. ePrint Arch.*, 2016.
- [37] S. Pinto and N. Santos, “Demystifying ARM TrustZone: A comprehensive survey,” *ACM Computing Surveys*, 2019.
- [38] Y. Sun et al., “Trust region policy optimization for LLM alignment,” in *Proc. ICML*, 2023.
- [39] D. Amodei et al., “Concrete problems in AI safety,” *arXiv preprint arXiv:1606.06565*, 2016.
- [40] R. Bost et al., “Machine learning classification over encrypted data,” in *Proc. NDSS*, 2015.
- [41] S. Badrinarayanan et al., “Verifiable capacity from zero knowledge proofs,” in *Proc. TCC*, 2020.

- [42] J. H. Ahn et al., “RISC Zero zkVM: Scalable, transparent zero-knowledge proofs,” *Technical Report*, RISC Zero, 2022.
- [43] H. Kalodner et al., “SP1: A high-performance scalable zkVM,” *Technical Report*, Succinct Labs, 2024.
- [44] A. Arun, S. Setty, and J. Thaler, “Jolt: SNARKs for Virtual Machines via Lookups,” in *Proc. EUROCRYPT*, 2024.
- [45] Y. Lu et al., “PipeMSM: Hardware acceleration for multi-scalar multiplication,” in *Proc. IEEE FCCM*, 2024.
- [46] J. Li et al., “EZKL: Verified inference for computational graphs using zero-knowledge proofs,” *arXiv preprint arXiv:2309.05344*, 2023.
- [47] Giza, “Verifiable machine learning with STARK proofs on StarkNet,” *Technical Report*, Giza Technologies, 2023.